A. Informasi & Ringkasan

1. Identitas

NIM	11S25038
Nama	EVAN BENTA PURBA
Kelas	11IF-2
Judul Praktikum	Tipe Bentukan : Point

2. Capaian & Ringkasan Pemahaman

Hasil Capaian setelah mengikuti praktikum:

- A (Penyelesaian): Selesai (1), Tidak Selesai (0).
- B (Pemahaman): Tidak Paham (1), Kurang Paham (2), Cukup Paham (3), Paham (4), Sangat Paham (5).

no	Kriteria	A	B
1	Menuliskan Notasi Fungsional Tipe Bentukan	1	5
2	Mengimplementasikan tipe bentukan	1	4
3	Melengkapi Fungsi Predikat dan Operator pada Tipe Point	1	4

3. Foto Praktikan



B. Penilaian Observasi

Setiap Mahasiswa yang mengikuti kegiatan praktikum perlu melakukan penilaian terhadap diri sendiri dan rekan kerja berdasarkan [Indikator Penilaian Observasi Praktikum].

1. Penilaian Terhadap Diri Sendiri

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	4
3	Kerja Sama Tim	4
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	5

2. Penilaian Terhadap Rekan Kerja

Harus melakukan penilaian terhadap rekan kerja selama aktivitas praktikum berlangsung.

a) Steven manurung (11S25013)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	4
2	Kemampuan Penyelesaian Tugas	2
3	Kerja Sama Tim	5
4	Inisiatif dan Problem Solving	1
5	Kedisiplinan dan Etika Kerja	5

b) Felip Hutasoit (11S25037)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	3
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	5

c) Deon Hutagalung (11S25039)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	2
4	Inisiatif dan Problem Solving	4
5	Kedisiplinan dan Etika Kerja	5

C. Laporan Aktivitas Praktikum

1. Menuliskan Notasi Fungsional Tipe Bentukan

Kode Program "tipePoint.hs" Pada bagian ini kamu akan mempelajari cara menuliskan notasi fungsional. Berikut merupakan cara menuliskan notasi fungsional untuk tipe point.

TYPE POINT
DEFINISI TYPE type point : < x: integer, y:integer > { adalah sebuah point (titik) dalam kordinat kartesius dengan x adalah absis dan y adalah ordinat }
DEFINISI DAN SPESIFIKASI SELEKTOR absis : Point → integer { absis(p) memberikan absis dari Point p } ordinat : Point → integer { ordinat(p) memberikan ordinat dari Point p }
DEFINISI DAN SPESIFIKASI KONSTRUKTOR makePoint : 2 integer → Point { makePoint(x,y) membentuk sebuah Point p dari x dan y dengan x sebagai absis dan y sebagai ordinat }
DEFINISI DAN SPESIFIKASI PREDIKAT isOrigin : Point → boolean { isOrigin(p) mengembalikan nilai benar jika titik p adalah titik pusat (0,0) } kuadran : point → integer { kuadran (p) mengembalikan lokasi kuadran dari titik p }

Analisis tahapan dari notasi fungsional di atas.

[!] Silahkan membaca dan mempelajari diktat untuk penulisan notasi fungsional lebih lanjut.

Analisis:
Tipe point didefinisikan sebagai sebuah struktur dengan dua atribut:

- x: merepresentasikan **absis** (koordinat horizontal) yang bertipe integer.
- y: merepresentasikan **ordinat** (koordinat vertikal) yang juga bertipe integer.
- **absis : Point → integer**
  Fungsi ini mengembalikan nilai absis dari titik p. Jika p adalah makePoint(3, 4), maka absis(p) akan mengembalikan 3.
- **ordinat : Point → integer**
  Fungsi ini mengembalikan nilai ordinat dari titik p. Untuk makePoint(3, 4), ordinat(p) mengembalikan 4.

- **makePoint : 2 integer → Point**
  Fungsi ini menerima dua nilai integer (x dan y), kemudian membentuk sebuah objek bertipe point dengan x sebagai absis dan y sebagai ordinat. Misalnya, makePoint(3, 4) menghasilkan titik (3, 4).
- **isOrigin : Point → Boolean**
  Fungsi ini mengembalikan true jika titik p berada di pusat koordinat (0, 0). Sebagai contoh, isOrigin(makePoint(0, 0)) mengembalikan true, sedangkan isOrigin(makePoint(1, 2)) mengembalikan false.
- **kuadran : Point → integer**
  Fungsi ini mengembalikan kuadran di mana titik p berada pada sistem koordinat kartesius. Sistem kartesius memiliki empat kuadran, dan fungsi ini mengidentifikasi lokasi titik berdasarkan nilai x dan y. Sebagai contoh:

  Jika x > 0 dan y > 0, maka titik berada di **Kuadran I.**
  Jika x < 0 dan y > 0, maka titik berada di **Kuadran II.**
  Jika x < 0 dan y < 0, maka titik berada di **Kuadran III.**
  Jika x > 0 dan y < 0, maka titik berada di **Kuadran IV.**

Dengan Kesimpulan :

Kode ini menyediakan definisi tipe data point beserta fungsi fungsi terkait untuk membuat, mengakses, dan menganalisis titik dalam sistem koordinat kartesius. Struktur ini berguna dalam berbagai aplikasi, seperti pemetaan, grafika komputer, atau perhitungan geometri.

Mengimplementasikan Tipe Bentukan

Pada bagian ini kamu akan mempraktikan dan mempelajari cara merealisasikan notasi fungsional ke bahasa pemrograman Haskell.
Kode Program ”tipePoint.hs”

tipePoint.hs

```haskell
-- Definisi tipe data Point
data Point = Point { x :: Int, y :: Int } deriving Show

-- realisasi selektor
absis :: Point -> Int
absis point = x point

ordinat :: Point -> Int
ordinat point = y point

-- realisasi konstruktor
makePoint :: Int -> Int -> Point
makePoint x y = Point { x = x, y = y }

-- realisasi predikat
isOrigin :: Point -> Bool
isOrigin p = absis p == 0 && ordinat p == 0

kuadran :: Point -> String
kuadran p
  | absis p > 0 && ordinat p > 0 = "1"
  | absis p < 0 && ordinat p > 0 = "2"
  | absis p < 0 && ordinat p < 0 = "3"
  | absis p > 0 && ordinat p < 0 = "4"
  | otherwise                    = "Lainnya"

```

```haskell
main :: IO ()
main = do
  -- membuat point
  let p1 = makePoint 2 (-2)
      p2 = makePoint 2 2
      p3 = makePoint 0 0
      p4 = makePoint (-2) (-2)
      p5 = makePoint (-2) 2

  print (isOrigin p1)
  print (isOrigin p2)
  print (isOrigin p3)
  print (isOrigin p4)
  print (isOrigin p5)

  putStrLn (kuadran p1)
  putStrLn (kuadran p2)
  putStrLn (kuadran p3)
  putStrLn (kuadran p4)
  putStrLn (kuadran p5)
```

Silahkan melakukan compile & run program secara mandiri.

Run Mandiri :

tipePoint.hs

```haskell
-- Definisi tipe data Point
data Point = Point { x :: Int, y :: Int } deriving Show

 -- realisasi selektor
absis :: Point -> Int
absis = x

ordinat :: Point -> Int
ordinat = y

 -- realisasi konstruktor
makePoint :: Int -> Int -> Point
makePoint x y = Point { x = x, y = y }

 -- realisasi predikat
isOrigin :: Point -> Bool
isOrigin p = absis p == 0 && ordinat p == 0

kuadran :: Point -> String
kuadran p
  | absis p > 0 && ordinat p > 0 = "1"
  | absis p < 0 && ordinat p > 0 = "2"
  | absis p < 0 && ordinat p < 0 = "3"
  | absis p > 0 && ordinat p < 0 = "4"
  | otherwise                    = "Lainnya"

main :: IO ()
main = do
  -- membuat point
  let p1 = makePoint 2 (-2)
      p2 = makePoint 2 2
      p3 = makePoint 0 0
      p4 = makePoint (-2) (-2)
      p5 = makePoint (-2) 2

  print (isOrigin p1)
  print (isOrigin p2)
  print (isOrigin p3)
  print (isOrigin p4)
  print (isOrigin p5)

  putStrLn (kuadran p1)
  putStrLn (kuadran p2)
  putStrLn (kuadran p3)
  putStrLn (kuadran p4)
  putStrLn (kuadran p5)
```

```
C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\{ifs25038} - pemfung-praktikum-5>ghc tipePoint.hs
[1 of 2] Compiling Main             ( tipePoint.hs, tipePoint.o )
[2 of 2] Linking tipePoint.exe

C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\{ifs25038} - pemfung-praktikum-5>tipePoint.exe
False
False
True
False
False
4
1
Lainnya
3
2
```

Realisasi notasi fungsional dapat dilengkapi seperti berikut:

```
TYPE POINT
Definisi Type
Type point :  <x: Integer, y: Integer>
   { <x,y> adalah sebuah point (titik)dalam koordinat
kartesisus dengan x adalah absis dan y adalah ordinat }
Realisasi Haskell untuk Definisi Type Point
 Data point = Point { x :: Int, y :: Int } deriving show
DEFINISI DAN SPESIFIKASI SELEKTOR
absis : Point → integer
{ absis(p) memberikan absis dari Point p }
ordinat : Point → integer
{ ordinat(p) memberikan ordinat dari Point p }
REALISASI HASKELL UNTUK SELEKTOR
absis :: Point -> Int
absis point = x point

ordinat :: Point -> Int
ordinat point = y point
DEFINISI DAN SPESIFIKASI KONSTRUKTOR
makePoint : 2 integer → Point
 { makePoint(x,y) membentuk sebuah Point p dari x dan y
dengan
x sebagai absis dan y sebagai ordinat }
REALISASI HASKELL UNTUK KONSTRUKTOR
makePoint :: Int -> Int -> Point
makePoint x y = Point { x = x, y = y }
DEFINISI DAN SPESIFIKASI PREDIKAT
isOrigin : Point → boolean
 { isOrigin(p) mengembalikan nilai benar jika titik p adalah
titik pusat (0,0) }
kuadran : point → integer
 { kuadran (p) mengembalikan lokasi kuadran dari titik p }
REALISASI HASKELL UNTUK PREDIKAT
isOrigin :: Point -> Bool
isOrigin p = absis p == 0 && ordinat p == 0
kuadran :: Point -> String
kuadran p
| absis p > 0 && ordinat p > 0 = "1"
| absis p < 0 && ordinat p > 0 = "2"
| absis p < 0 && ordinat p < 0 = "3"
| absis p > 0 && ordinat p < 0 = "4"
| otherwise = "Lainnya"
APLIKASI
⇒ (isOrigin (makePoint 2 (-2)))
False
⇒ (isOrigin (makePoint 2 2))
```

```
False
⇒ (isOrigin (makePoint 0 0))
True
⇒ (isOrigin (makePoint (-2) (-2)))
False
⇒ (isOrigin (makePoint (-2) 2))
False
⇒ (kuadran (makePoint 2 (-2)))
4
⇒ (kuadran (makePoint 2 2))
1
⇒ (kuadran (makePoint 0 0))
Lainnya
⇒ (kuadran (makePoint (-2) (-2)))
3
⇒ (kuadran (makePoint (-2) 2))
2
```

2. Melengkapi Fungsi Predikat dan Operator pada Tipe Point

Lengkapi fungsi pada tipe bentukan Point. Silahkan membuat notasi fungsional lengkapnya sampai terdapat realisasi Haskell dan aplikasi pada Laporan Praktikum. Fungsi yang harus anda tambahkan adalah:

1) **isOnSbY(p)**
   isOnSbY(p) adalah fungsi untuk melakukan pengecekan apakah titik p berada pada sumbu Y.
2) **isEqual(p1, p2)**
   isEqual(p1, p2) adalah fungsi untuk melakukan pengecekan apakah titik p1 sama dengan titik p2.
3) **translasiSbY(p, n)**
   translasiSbY(p, n) adalah fungsi untuk mengembalikan sebuah titik hasil translasi titik p, searah sumbu Y, sejauh n.
4) **jarak(p1, p2)**
   jarak(p1, p2) adalah fungsi untuk mengembalikan sebuah nilai real, merupakan jarak antara titik p1 dan p2

Kode program "**tipePoint.hs**", seperti berikut:

tipePoint.hs

```
-- Definisi tipe data Point
-- > <x,y> adalah sebuah point (titik) dalam kordinat kartesius dengan
x adalah absis dan y adalah ordinat
data Point = Point { x :: Int, y :: Int } deriving Show

-- Realisasi Selektor
-- > absis(p) memberikan absis dari Point p
absis :: Point -> Int
absis point = x point
-- > ordinat(p) memberikan ordinat dari Point p
ordinat :: Point -> Int
ordinat point = y point

-- Realisasi Konstruktor
-- > makePoint(x,y) membentuk sebuah Point p dari x dan y dengan x
sebagai absis dan y sebagai ordinat
makePoint :: Int -> Int -> Point
makePoint x y = Point { x = x, y = y }

-- Realisasi Predikat
-- > isOrigin(p) mengembalikan nilai benar jika titik p adalah titik
pusat (0,0)
isOrigin :: Point -> Bool
isOrigin p = absis p == 0 && ordinat p == 0
```

```
-- > kuadran (p) mengembalikan lokasi kuadran dari titik p
kuadran :: Point -> String
kuadran p
  | absis p > 0 && ordinat p > 0 = "1"
  | absis p < 0 && ordinat p > 0 = "2"
  | absis p < 0 && ordinat p < 0 = "3"
  | absis p > 0 && ordinat p < 0 = "4"
  | otherwise                    = "Lainnya"
-- > isOnSbY(p) adalah fungsi untuk melakukan pengecekan apakah titik p
berada pada sumbu Y.
isOnSbY :: Point -> Bool
-- TODO: realisasi Haskell disini

-- >isEqual(p1, p2) adalah fungsi untuk melakukan pengecekan apakah
titik p1 sama dengan titik p2.
isEqual :: Point -> Point -> Bool
-- TODO: realisasi Haskell disini

-- Realisasi Operator
-- > translasiSbY(p, n) adalah fungsi untuk mengembalikan sebuah titik
hasil translasi titik p, searah sumbu Y, sejauh n.
translasiSbY :: Point -> Int -> Point
-- TODO: realisasi Haskell disini

-- >jarak(p1, p2) adalah fungsi untuk mengembalikan sebuah nilai real,
merupakan jarak antara titik p1 dan p2
jarak :: Point -> Point -> Double
-- TODO: realisasi Haskell disini


main :: IO ()
main = do
  x1 <- readLn :: IO Int
  y1 <- readLn :: IO Int
  x2 <- readLn :: IO Int
  y2 <- readLn :: IO Int
  n <- readLn :: IO Int

  -- membuat point
  let p1 = makePoint x1 y1
      p2 = makePoint x2 y2

  putStrLn $ "p1 = " ++ show p1
  putStrLn $ "p2 = " ++ show p2
  putStrLn $ "n = " ++ show n

  -- memanggil fungsi isOnSbY(p)
```

```
  putStrLn $ "isOnSbY(p1) = " ++ show (isOnSbY p1)
  putStrLn $ "isOnSbY(p2) = " ++ show (isOnSbY p2)

  -- memanggil fungsi isEqual(p1, p2)
  putStrLn $ "isEqual(p1, p2) = " ++ show (isEqual p1 p2)

  -- memanggil fungsi translasiSbY(p, n)
  putStrLn $ "translasiSbY(p1, n) = " ++ show (translasiSbY p1 n)
  putStrLn $ "translasiSbY(p2, n) = " ++ show (translasiSbY p2 n)

  -- memanggil fungsi jarak(p1, p2)
  putStrLn $ "jarak(p1, p2) = " ++ show (jarak p1 p2)
```

Hasil yang diharapkan sebagai berikut :

Input	Output
10 1 8 1 1	p1 = Point {x = 10, y = 1} p2 = Point {x = 8, y = 1} n = 1 isOnSbY(p1) = False isOnSbY(p2) = False isEqual(p1, p2) = False translasiSbY(p1, n) = Point {x = 10, y = 2} translasiSbY(p2, n) = Point {x = 8, y = 2} jarak(p1, p2) = 2.0

Compile dan run:

tipePoint.hs

```haskell
-- Definisi tipe data Point -- > <x,y> adalah sebuah point (titik) dalam kordinat kartesius dengan
-- x adalah absis dan y adalah ordinat
data Point = Point { x :: Int, y :: Int } deriving Show
 -- Realisasi Selektor -- > absis(p) memberikan absis dari Point p
absis :: Point -> Int
absis = x -- > ordinat(p) memberikan ordinat dari Point p
ordinat :: Point -> Int
ordinat = y
 -- Realisasi Konstruktor -- > makePoint(x,y) membentuk sebuah Point p dari x dan y dengan x
-- sebagai absis dan y sebagai ordinat
makePoint :: Int -> Int -> Point
makePoint x y = Point { x = x, y = y }
 -- Realisasi Predikat -- > isOrigin(p) mengembalikan nilai benar jika titik p adalah titik
-- pusat (0,0)
isOrigin :: Point -> Bool
isOrigin p = absis p == 0 && ordinat p == 0
-- > kuadran (p) mengembalikan lokasi kuadran dari titik p
kuadran :: Point -> String
kuadran p
  | absis p > 0 && ordinat p > 0 = "1"
  | absis p < 0 && ordinat p > 0 = "2"
  | absis p < 0 && ordinat p < 0 = "3"
  | absis p > 0 && ordinat p < 0 = "4"
  | otherwise                    = "Lainnya"
-- > isOnSbY(p) adalah fungsi untuk melakukan pengecekan apakah titik p berada pada sumbu Y.
isOnSbY :: Point -> Bool
isOnSbY p = absis p == 0
-- >isEqual(p1, p2) adalah fungsi untuk melakukan pengecekan apakah titik p1 sama dengan titik p2.
isEqual :: Point -> Point -> Bool
isEqual p1 p2 = absis p1 == absis p2 && ordinat p1 == ordinat p2
-- Realisasi Operator -- > translasiSbY(p, n) adalah fungsi untuk mengembalikan sebuah titik hasil translasi titik p, searah sumbu Y, sejauh n.
translasiSbY :: Point -> Int -> Point
translasiSbY p n = makePoint (absis p) (ordinat p + n)
-- >jarak(p1, p2) adalah fungsi untuk mengembalikan sebuah nilai real, merupakan jarak antara titik p1 dan p2
jarak :: Point -> Point -> Double
jarak p1 p2 = sqrt (dx * dx + dy * dy)
  where
    dx = fromIntegral (absis p2 - absis p1)
    dy = fromIntegral (ordinat p2 - ordinat p1)

main :: IO ()
main = do
  x1 <- readLn :: IO Int
  y1 <- readLn :: IO Int
  x2 <- readLn :: IO Int
  y2 <- readLn :: IO Int
  n <- readLn :: IO Int

  -- membuat point
  let p1 = makePoint x1 y1
      p2 = makePoint x2 y2
```

```haskell
    p2 = makePoint x2 y2

  putStrLn $ "p1 = " ++ show p1
  putStrLn $ "p2 = " ++ show p2
  putStrLn $ "n = " ++ show n

  -- memanggil fungsi isOnSbY(p)
  putStrLn $ "isOnSbY(p1) = " ++ show (isOnSbY p1)
  putStrLn $ "isOnSbY(p2) = " ++ show (isOnSbY p2)

  -- memanggil fungsi isEqual(p1, p2)
  putStrLn $ "isEqual(p1, p2) = " ++ show (isEqual p1 p2)

  -- memanggil fungsi translasiSbY(p, n)
  putStrLn $ "translasiSbY(p1, n) = " ++ show (translasiSbY p1 n)
  putStrLn $ "translasiSbY(p2, n) = " ++ show (translasiSbY p2 n)

  -- memanggil fungsi jarak(p1, p2)
  putStrLn $ "jarak(p1, p2) = " ++ show (jarak p1 p2)
```

```
C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\{ifs25038} - pemfung-praktikum-5>ghc tipePoint.hs
[1 of 2] Compiling Main             ( tipePoint.hs, tipePoint.o )
[2 of 2] Linking tipePoint.exe

C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\{ifs25038} - pemfung-praktikum-5>tipePoint.exe
10
1
8
1
1
p1 = Point {x = 10, y = 1}
p2 = Point {x = 8, y = 1}
n = 1
isOnSbY(p1) = False
isOnSbY(p2) = False
isEqual(p1, p2) = False
translasiSbY(p1, n) = Point {x = 10, y = 2}
translasiSbY(p2, n) = Point {x = 8, y = 2}
jarak(p1, p2) = 2.0

C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\{ifs25038} - pemfung-praktikum-5>
```